

Lesson 8: Deploying to Vercel and Sharing

Mission Brief

A ship built in the hangar isn't a ship until it launches. All the HTML and CSS you've written so far has only ever lived on your own computer — today, Navigator, you complete the mission: you ship your work to a real server on the internet, get a real public URL, and share your project with the galaxy.

Learning Objectives

- Explain what a static site host is and why Vercel suits HTML/CSS projects
- Initialize a local Git repository and make a first commit
- Push a project to GitHub
- Deploy a static site to Vercel via both the website import flow and the Vercel CLI
- Share a live deployed URL and understand automatic redeloys on new commits

What You'll Build

A live, public URL for your Crew Profile page (or any project from this course), deployed on Vercel and reachable by anyone with the link.

Prerequisites

Lessons 1-7: a complete, working multi-file HTML/CSS project on your computer.

Key Concepts

- Version control / Git
- Repository, commit, push
- GitHub
- Static site hosting
- Vercel project import
- Vercel CLI (`vercel` command)
- Continuous deployment

Concept Explanation

Until now, your pages have only existed as files on your own hard drive — opening `index.html` directly in a browser uses the `file://` protocol, not a real server. To make a site reachable by anyone in the world at a real URL, you need to **deploy** it to a hosting service. Because your projects in this course are pure HTML/CSS/minimal-JS with no build step or backend server required, they qualify as **static sites** — files served exactly as written, with no server-side processing. Vercel is a popular, free-tier-friendly platform built exactly for deploying static sites (and much more) with almost zero configuration.

The standard professional workflow uses **Git**, a version control system that tracks changes to your files over time, and **GitHub**, a website that hosts Git repositories online. Here's the process from scratch:

Step 1 — Initialize Git locally. In your project folder, run:

```
git init
git add .
git commit -m "Initial commit: Crew Profile site"
```

`git init` creates a new repository in the current folder. `git add .` stages all your files for saving. `git commit -m "..."` saves a permanent snapshot (a "commit") with a descriptive message.

Step 2 — Push to GitHub. Create a new, empty repository on github.com (no README, no .gitignore — keep it empty so it doesn't conflict with your existing commit). GitHub will show you commands like:

```
git remote add origin https://github.com/yourusername/crew-profile.git
git branch -M main
git push -u origin main
```

This connects your local repository to the empty one on GitHub (`remote add`), renames your default branch to `main` , and uploads (`push`) your commit history.

Step 3 — Deploy with Vercel, Option A: the website. Go to vercel.com, sign in (GitHub sign-in is easiest since it can then read your repos), click "Add New Project," and select your `crew-profile` repository. Since this is a plain static site, Vercel auto-detects there's no build command needed — just click "Deploy." Within seconds you'll get a live URL like `crew-profile.vercel.app` .

Step 3 — Deploy with Vercel, Option B: the CLI. Install the Vercel command-line tool (`npm install -g vercel` , which requires Node.js to be installed), then from your project folder simply run:

```
vercel
```

The CLI will ask a few setup questions (link to existing project or create new, project name, which directory to deploy) and then upload your files directly, giving you a live URL immediately — no GitHub required for this path, though connecting GitHub is still recommended for automatic redeployments.

The real power of connecting Vercel to GitHub is **continuous deployment**: every time you `git push` a new commit to your repository's main branch, Vercel automatically detects it and redeploys your live site within seconds

— you never manually re-upload files again. This is exactly how professional teams ship updates.

Guided Coding

```
# From inside your project folder (containing index.html, styles.css)

# 1. Initialize version control
git init
git add .
git commit -m "Initial commit: Crew Profile site"

# 2. Connect to GitHub (after creating an empty repo on github.com)
git remote add origin https://github.com/yourusername/crew-profile.git
git branch -M main
git push -u origin main

# 3a. Deploy via Vercel CLI (alternative to the website import)
npm install -g vercel
vercel
# Answer the CLI prompts, then Vercel prints your live URL, e.g.:
# https://crew-profile-yourusername.vercel.app

# 4. Every future update:
git add .
git commit -m "Update skills section"
git push
# Vercel automatically redeploys within seconds – no extra steps needed
```

```
<!-- A good practice: add a small footer credit + live-status note before deploying -->
<footer>
  <p>&copy; 2026 Nova Ortiz. Built with HTML & CSS. Deployed on Vercel.</p>
</footer>
```

Code Walkthrough

- `git init` creates the hidden `.git` folder that tracks your project's history — run this once, at the very start.
- `git add .` stages every changed file in the current folder (the `.` means "everything here").
- `git commit -m "..."` permanently records a snapshot with a human-readable message describing what changed.
- `git remote add origin <url>` tells your local repo where its "remote" copy lives on GitHub.
- `git push -u origin main` uploads your commits to GitHub; the `-u` flag remembers this pairing so future pushes can just be `git push`.

- `vercel` (the CLI command) reads your project folder, uploads it, and returns a public HTTPS URL — Vercel automatically serves `index.html` as your homepage.
- Subsequent `git push` commands after connecting GitHub to Vercel trigger automatic redeloys — this is continuous deployment in action.

Student Mission

1. Make sure your Crew Profile project folder contains a working `index.html` and `styles.css` (rename `mission-log.html` or `crew-profile.html` to `index.html` if needed — Vercel serves `index.html` as the homepage by default).
2. Run `git init`, `git add .`, and `git commit -m "Initial commit"` in that folder.
3. Create a new empty repository on GitHub and push your project to it.
4. Import that repository on `vercel.com` and deploy it, OR install and run the `vercel` CLI directly.
5. Confirm your live URL loads correctly in an incognito/private browser window (proving it's really public, not just cached locally).

Challenge

Make a small visible change to your page (e.g. update a heading), commit it with a clear message, push it to GitHub, and time how long it takes for the live Vercel URL to reflect the update automatically. Document the elapsed time in a comment.

Experiment/Extension

Try deploying the exact same project a second time as a brand-new Vercel project with a different project name, and compare the two resulting URLs. Note that Vercel lets you attach a custom domain to any project later, though that's beyond this lesson's scope.

Common Mistakes

- Forgetting to rename your main file to exactly `index.html` (Vercel/most static hosts look for this filename specifically as the default page).
- Creating the GitHub repository with a README or `.gitignore` already checked, causing a merge conflict on first push (fixable, but confusing for beginners — best avoided by creating it empty).
- Using absolute local file paths (like `C:\Users\name\project\styles.css`) instead of relative paths (`styles.css` or `./styles.css`) in `<link>` / `<img>` tags — these will work locally but break completely once deployed.
- Forgetting to `git add` / `commit` / `push` after making changes, then wondering why the live site didn't update.
- Not testing the deployed URL in an incognito window, potentially mistaking a locally cached old version for the real live state.

Debugging Tips

- If Vercel shows a 404 on your live URL, check that `index.html` is at the root of the deployed folder, not nested inside an extra subfolder.
- Use `git status` locally to see which files are staged, unstaged, or untracked before committing — this catches "I forgot to add a file" mistakes early.
- On Vercel's dashboard, each deployment has a "Build Logs" and "Source" view — use it to confirm exactly which files were actually uploaded.
- If a `<link>` or `<img>` breaks only after deploying (but worked locally), it's almost always an absolute local path — switch to relative paths.

Check Your Understanding

1. What is the difference between `git commit` and `git push`?
2. Why must your homepage file be named `index.html` for most static hosts?
3. What are the two ways covered in this lesson to deploy a static site to Vercel?
4. What is "continuous deployment," and how does connecting GitHub to Vercel enable it?
5. Why should file paths in your HTML/CSS be relative rather than absolute?

Mini Quiz + Answer Key

Quiz 1. Which command uploads your local commits to GitHub? a) `git commit` b) `git push` c) `git add` d) `git init` 2. True or False: Vercel requires a build step for plain HTML/CSS sites. 3. What CLI command deploys the current folder directly to Vercel? 4. What filename does Vercel serve by default as a site's homepage? 5. What triggers an automatic redeploy once GitHub is connected to Vercel?

Answer Key 1. b) `git push` 2. False — plain static HTML/CSS needs no build step. 3. `vercel` 4. `index.html` 5. A new `git push` to the connected repository's main branch.

Lesson Recap

- Git tracks your project's history locally; GitHub hosts that history online.
- Vercel deploys static sites either via GitHub import or the `vercel` CLI directly.
- `index.html` must be your homepage filename.
- Connecting GitHub to Vercel enables automatic red deploys on every push.
- Always use relative file paths so links/styles/images work identically locally and once deployed.

Homework

Deploy your complete Crew Profile page to Vercel following every step in this lesson, and write down (as a note, or in your project's README) the final live URL plus the date you deployed it.

Portfolio Project Connection

This lesson's deployment workflow is the final required step for **all three course projects** — each project's README documents exactly these Vercel deployment steps so Project 01, 02, and 03 can all go live as real, shareable, public URLs.